Ascent
FORMATION

FORMATION PROFESSIONNELLE · COURS APPRENANT

Cours apprenant

ReST API — bonnes pratiques et sécurité

MODULE 4

Pratiques avancées de sécurisation

Permissions fines, chiffrement TLS, HSTS, sessions, cookies, KMS

CRÉATEUR DU COURS
Boris Rose
Formateur & concepteur du contenu

Table des matières

Avant-propos **2**

1 Modèles d'autorisation **2**
1.1 RBAC — Role-Based Access Control . . . 2
1.2 ABAC — Attribute-Based Access Control . . . 2
1.3 ReBAC — Relationship-Based Access Control . . . 2
1.4 Architecture canonique d'autorisation . . . 2
1.5 Principes transverses . . . 3
1.5.1 Vérification au niveau objet : ce que cela signifie concrètement . . . 4

2 TLS : chiffrement en transit **5**
2.1 Versions et état 2026 . . . 5
2.2 Cipher suites et Forward Secrecy . . . 5
2.3 Certificats X.509 . . . 5
2.4 ACME et Let's Encrypt . . . 6
2.5 Configuration HTTPS et HSTS . . . 6
2.6 Headers de sécurité à activer . . . 6

3 Sessions et cookies **7**
3.1 Cycle de vie d'une session sécurisée . . . 7
3.1.1 Création d'une session — en détail . . . 7
3.1.2 Durée absolue — pourquoi et exemple . . . 8
3.2 Flags de cookies à connaître . . . 9
3.2.1 Le préfixe `__Host-` — la défense en profondeur des cookies . . . 9

4 Chiffrement au repos **10**
4.1 Trois niveaux distincts . . . 10
4.1.1 LUKS, FileVault, TDE — les chiffrements de stockage . . . 10
4.2 Envelope encryption . . . 11
4.3 Algorithmes recommandés . . . 12
4.3.1 Les quatre algorithmes de hash de mot de passe — exemples . . . 12

5 Les outils du module **13**
5.1 Certbot . . . 13
5.2 nginx . . . 15
5.3 testssl.sh . . . 15
5.4 Qualys SSL Labs . . . 15
5.5 Mozilla SSL Configuration Generator . . . 15
5.6 HashiCorp Vault, AWS Secrets Manager . . . 15

6 Synthèse **16**

7 Lexique du module **16**

8 Pour aller plus loin **17**

Avant-propos

Ce cours est le compagnon écrit du **Module 4**. Il couvre les modèles d'autorisation, la configuration TLS de l'état de l'art, la gestion sécurisée des sessions et des cookies, et les outils utilisés en TP. *Cours conçu et rédigé par Boris Rose, formateur.*

1 Modèles d'autorisation

1.1 RBAC — Role-Based Access Control

Définition

RBAC : modèle classique où l'utilisateur reçoit des *rôles*, et chaque rôle dispose d'un ensemble de *permissions*. La décision est : « cet utilisateur a-t-il un rôle qui inclut cette permission ? ».

Forces. Lisibilité, simplicité, alignement naturel avec une organisation hiérarchique. Standardisé par le NIST.

Limites. Explosion combinatoire dès que les permissions doivent dépendre du contexte (par projet, par client). Le mythique *role explosion* : finir avec 200 rôles distincts.

Quand l'utiliser ? 90 % des produits SaaS commencent en RBAC simple. Suffit dans la plupart des cas.

1.2 ABAC — Attribute-Based Access Control

Définition

ABAC : la décision est calculée à partir d'*attributs* : ceux de l'utilisateur (rôle, département), de la ressource (propriétaire, classification), de l'action et du contexte (heure, IP).

Quand l'utiliser ? Quand le besoin de finesse dépasse ce que RBAC peut exprimer raisonnablement (grandes orgs, conformité, permissions contextuelles type *break the glass*).

1.3 ReBAC — Relationship-Based Access Control

Définition

ReBAC : la décision est basée sur les *relations* entre objets et utilisateurs (Alice est éditrice du document X, le document X appartient au workspace Y, Bob est membre de Y). Modèle popularisé par le système Zanzibar de Google.

Quand l'utiliser ? Apps collaboratives où les permissions suivent les relations entre objets (Google Docs, Notion, Linear).

1.4 Architecture canonique d'autorisation

Les modèles RBAC, ABAC et ReBAC partagent tous la même architecture en quatre composants. Cette séparation est issue du standard XACML.

- **PEP** (*Policy Enforcement Point*) : composant qui **intercepte** les requêtes et applique la décision. Implémenté typiquement comme un middleware HTTP au sein de l'API.

- **PDP** (*Policy Decision Point*) : moteur qui répond *oui* ou *non*. Reçoit une question (« user 42 peut-il `PATCH /orders/7` ? ») et retourne une décision. Exemples : OPA (Open Policy Agent), Cedar (AWS).
- **PIP** (*Policy Information Point*) : sources d'information consultées par le PDP pour décider (rôles utilisateur dans la base, attributs IdP, propriétaire d'une ressource).
- **PAP** (*Policy Administration Point*) : interface d'administration où l'on rédige, versionne et déploie les politiques d'autorisation.

Exemple

Scénario concret : un utilisateur essaie de modifier une commande.

1. L'utilisateur Alice envoie `PATCH /orders/7 { "status": "cancelled" }`.
2. Le **PEP** (middleware Express dans l'API) intercepte la requête *avant* le contrôleur métier.
3. Le PEP construit une question : « `subject=alice, action=update, resource=orders/7` » et l'envoie au **PDP**.
4. Le **PDP** (instance OPA tournant en sidecar) évalue la politique Rego suivante :

```
package authz
default allow = false
allow {
  input.action == "update"
  input.resource.owner == input.subject.id # Alice est-elle propriétaire ?
  input.resource.status != "shipped" # commande non encore expédiée ?
}
```

5. Pour évaluer, le PDP appelle le **PIP** : une requête SQL `SELECT owner, status FROM orders WHERE id=7` renvoie `(owner=42, status="pending")`. Alice a l'id 42.
6. Décision : `allow=true`. Le PEP laisse passer la requête.
7. Si Bob (id=99) tente la même requête, le PIP renverra le même `owner=42`, la condition échoue → `allow=false` → le PEP retourne `404 Not Found` (pas 403, pour ne pas révéler l'existence).
8. Le **PAP** est une console web où l'équipe sécurité a publié cette politique Rego, l'a versionnée dans Git et l'a déployée vers les OPA en production.

1.5 Principes transverses

À retenir

1. **Moindre privilège** : ne donner que les droits nécessaires à la tâche.
2. **Refus par défaut** : toute action non explicitement autorisée est interdite.
3. **Vérification à chaque endpoint**, pas un contrôle global.
4. **Vérification au niveau objet**, pas seulement par type de ressource (détaillé ci-dessous).
5. **Auditer chaque décision** : qui, quand, quoi, pourquoi.
6. **Revoir périodiquement** : audit trimestriel des accès dormants.

1.5.1 Vérification au niveau objet : ce que cela signifie concrètement

Définition

Vérification au niveau objet : ne pas se contenter de demander « cet utilisateur a-t-il le *rôle* pour effectuer ce type d'action ? » mais aussi « a-t-il le droit sur *cette instance précise* de l'objet ? ». Sans cette vérification, on tombe dans la faille BOLA (Broken Object Level Authorization, n°1 du Top OWASP API).

Différence avec la vérification au niveau du type.

- *Niveau type* (insuffisant seul) : « cet utilisateur a le rôle `Member`, donc il peut lire des commandes ».
- *Niveau objet* (nécessaire en plus) : « cet utilisateur a le rôle `Member` **et** la commande `#7` lui appartient (ou appartient à son organisation) ».

Exemple

Mauvaise pratique (BOLA) :

```
app.get("/orders/:id", requireRole("Member"), async (req, res) => {
  const order = await db.orders.findById(req.params.id);
  res.json(order); // n'importe quel Member voit n'importe quelle commande !
});
```

Bonne pratique (vérif niveau objet) :

```
app.get("/orders/:id", requireRole("Member"), async (req, res) => {
  const order = await db.orders.findOne({
    _id: req.params.id,
    ownerId: req.user.id // vérification niveau objet
  });
  if (!order) return res.status(404).end(); // 404, pas 403
  res.json(order);
});
```

Variante multi-tenant :

```
const order = await db.orders.findOne({
  _id: req.params.id,
  tenantId: req.user.tenantId // filtre par organisation
});
```

Variante avec partage explicite (Google Docs-style) :

```
const order = await db.orders.findOne({
  _id: req.params.id,
  $or: [
    { ownerId: req.user.id },
    { sharedWith: req.user.id },
    { tenantId: req.user.tenantId, "permissions.role": "admin" }
  ]
});
```

Mécanismes BDD complémentaires. Pour éviter d'oublier la vérification dans une nouvelle route, on peut imposer la sécurité au niveau de la base :

- *Row-Level Security* (PostgreSQL) : on attache une politique SQL qui filtre automatiquement chaque `SELECT/UPDATE/DELETE` par `owner = current_user`.
- *Tenant scoping* via un ORM (Sequelize *scopes*, Prisma *middleware*, Django *managers*) qui

injecte le filtre par défaut.

2 TLS : chiffrement en transit

Définition

TLS (*Transport Layer Security*, ex-SSL) : protocole de chiffrement qui assure trois propriétés sur HTTP :

- *Confidentialité* : personne ne peut lire le contenu en clair.
- *Intégrité* : personne ne peut modifier le contenu sans détection.
- *Authentification* : le client est sûr de parler au bon serveur (via le certificat).

2.1 Versions et état 2026

- SSL 2.0, 3.0 : cassés depuis 2014 (POODLE). **Désactiver**.
- TLS 1.0, 1.1 : dépréciés (RFC 8996, 2021). **Désactiver**.
- TLS 1.2 : acceptable, ciphers à choisir avec soin.
- TLS 1.3 : référence, ciphers réduits à 5 modernes, handshake plus court.

2.2 Cipher suites et Forward Secrecy

Définition

Forward Secrecy (PFS) : propriété qui garantit qu'une compromission ultérieure de la clé privée du serveur ne permettra pas de déchiffrer les sessions passées. Chaque session utilise une clé éphémère. Obtenue par les algorithmes `ECDHE`.

Configuration recommandée :

- Versions : TLS 1.3 (préféré) + TLS 1.2 (acceptable).
- Ciphers TLS 1.2 : ECDHE-ECDSA-AES128-GCM-SHA256 et équivalents.
- Forward secrecy : toujours ECDHE (pas de RSA en échange de clé).
- Certificat : ECDSA P-256 ou RSA 2048+, validation DNS.

2.3 Certificats X.509

- **DV** (*Domain Validated*) : preuve de contrôle du domaine. Let's Encrypt, ZeroSSL. Gratuit, instantané, suffisant pour la plupart des API.
- **OV** (*Organization Validated*) : preuve d'identité de l'organisation. Payant.
- **EV** (*Extended Validation*) : déprécié par les navigateurs.
- **Wildcard** : couvre `*.example.com`. Validation DNS requise.

Exemple

Comment fonctionne une validation DV ?

1. Vous demandez un certificat pour `api.example.com` à Let's Encrypt via `certbot`.
2. Let's Encrypt vous met au défi : « prouve-moi que tu contrôles `api.example.com` ». Il vous donne un *token* aléatoire, par exemple `aZ12...kP`.
3. Vous devez exposer ce token à une URL précise sur ce domaine, par exemple :

`http://api.example.com/.well-known/acme-challenge/aZ12...kP`
qui doit renvoyer `aZ12...kP.<empreinte>` (*challenge HTTP-01*).

4. Le serveur Let's Encrypt fait une requête HTTP vers cette URL. Si la bonne valeur revient, c'est la preuve que vous contrôlez le domaine (parce que vous savez configurer son serveur web).
5. Let's Encrypt vous délivre le certificat, valide 90 jours.

Variante DNS-01 (utilisée pour les certificats wildcards) : au lieu d'une URL HTTP, on doit publier un record TXT `_acme-challenge.api.example.com` contenant la valeur attendue. La preuve : on a le contrôle des DNS du domaine.

Coût total pour vous : 0 euro. Délai entre la commande et l'obtention : moins de 30 secondes en HTTP-01. Aucune intervention humaine côté CA.

Pourquoi c'est « suffisant pour la plupart des API » : la sécurité du transport est identique entre DV et OV (mêmes algorithmes, mêmes navigateurs). La différence d'un OV est uniquement la mention « *Issued by ... (verified organization)* » dans le détail du certificat, que personne ne consulte. Inutile pour une API REST.

2.4 ACME et Let's Encrypt

Définition

ACME (*Automated Certificate Management Environment*, RFC 8555) : protocole standard de demande et renouvellement automatique de certificats. Implémenté par Let's Encrypt, ZeroSSL, BuyPass.

Pourquoi automatiser ?

- Cycle de validité court (90 jours pour Let's Encrypt) → impossible à gérer manuellement à grande échelle.
- Un certificat oublié au renouvellement = downtime visible.

2.5 Configuration HTTPS et HSTS

Définition

HSTS (*HTTP Strict Transport Security*, RFC 6797) : en-tête HTTP qui force le navigateur à n'utiliser que HTTPS pour ce domaine, pendant une durée `max-age`.

Exemple

```
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

- `max-age=63072000` : 2 ans en secondes.
- `includeSubDomains` : applique à tous les sous-domaines.
- `preload` : permet l'inscription au *HSTS Preload List* de Chrome (`hstspreload.org`). Une fois acceptée, même la première connexion HTTP est interceptée.

2.6 Headers de sécurité à activer

- `Strict-Transport-Security` : HSTS.

- `X-Content-Type-Options: nosniff` : empêche le navigateur de deviner le MIME.
- `X-Frame-Options: DENY` : anti-clickjacking (legacy).
- `Content-Security-Policy: frame-ancestors 'none'` : remplaçant moderne.
- `Referrer-Policy: no-referrer` : limite la fuite via Referer.
- `Permissions-Policy` : contrôle l'accès aux APIs navigateur (camera, microphone).

3 Sessions et cookies

3.1 Cycle de vie d'une session sécurisée

1. **Création** : identifiant aléatoire fort ($\geq$ 128 bits) généré par CSPRNG, lié à l'utilisateur côté serveur (détaillé ci-dessous).
2. **Régénération à l'élévation** : après login, MFA ou changement de droits, nouvel ID, ancien invalidé.
3. **Inactivité** : la session expire après N minutes sans activité (15–30 min pour des données sensibles).
4. **Durée absolue** : la session expire après N heures, même si active (détaillé ci-dessous).
5. **Déconnexion** : invalidation côté serveur + suppression côté client.
6. **Multi-appareils** : permettre à l'utilisateur de lister et révoquer ses sessions distantes.

3.1.1 Création d'une session — en détail

Définition

CSPRNG (*Cryptographically Secure Pseudo-Random Number Generator*) : générateur de nombres pseudo-aléatoires dont la sortie est imprévisible même pour un attaquant qui observe une partie de la séquence. Repose sur des sources d'entropie matérielles (mouvements souris, températures CPU, instructions `RDRAND`). À ne pas confondre avec un PRNG classique (`Math.random()` en JS, `rand()` en C) dont les valeurs sont prévisibles — absolument inadapté à un identifiant de session.

Pourquoi 128 bits minimum ? Un identifiant à 128 bits offre $2^{128} \approx 3{,}4 \times 10^{38}$ valeurs possibles. Même avec un milliard de tentatives par seconde, deviner un identifiant valide prendrait plus de l'âge de l'univers. À 64 bits, ce serait encore faisable par un attaquant déterminé ; en-dessous, l'attaque devient triviale.

Pourquoi « lié à l'utilisateur côté serveur » ? L'identifiant de session est juste une clé opaque. La correspondance `session_id` → `user_id` doit vivre côté serveur (Redis ou table BDD), *jamais* dans le cookie. Cela permet :

- De révoquer instantanément une session (suppression de l'entrée serveur).
- D'éviter qu'un client modifie ses propres droits en triturant le cookie.
- De stocker des métadonnées (IP, user-agent, date d'expiration) sans les exposer au client.

Exemple

Implémentation en Node + Redis :

```
import { randomBytes } from "crypto";
import { redis } from "./redis.js";

async function createSession(userId, req) {
  // 32 octets = 256 bits d'entropie. Encodé en hex = 64 caractères.
```

```
  const sid = randomBytes(32).toString("hex");

  // Enregistrer côté serveur, lié àl'utilisateur
  await redis.hSet(`session:${sid}`, {
    userId,
    createdAt: Date.now(),
    absoluteExpireAt: Date.now() + 8 * 3600 * 1000, // 8h max
    ip: req.ip,
    userAgent: req.get("user-agent")
  });
  await redis.expire(`session:${sid}`, 30 * 60); // TTL inactivité 30 min

  return sid; // sera mis dans un cookie HttpOnly + Secure + SameSite
}
```

À ne PAS faire (PRNG faible) :

```
// MAUVAIS : Math.random() n'est PAS cryptographique
const sid = Math.random().toString(36).substring(2);
```

Pourquoi c'est cassé ? `Math.random()` utilise l'algorithme xorshift128+ qui peut être inversé après l'observation de quelques sorties consécutives. Un attaquant peut prédire les identifiants suivants.

3.1.2 Durée absolue — pourquoi et exemple

La **durée d'inactivité** (15–30 min) protège contre l'oubli sur un poste partagé. Mais elle peut être indéfiniment prolongée par des appels périodiques. La **durée absolue** pose une borne dure : passé ce délai, la session expire *quoi qu'il arrive*, l'utilisateur doit se reconnecter.

Valeurs typiques :

- Banque en ligne : durée absolue de 10–20 minutes (et inactivité 5 min).
- Application métier (CRM, ERP) : durée absolue de 8 heures (la journée de travail).
- Application grand public peu sensible (forum) : 7 jours, voire 30 jours avec *remember me*.

Exemple

Vérification combinée à chaque requête :

```
async function validateSession(sid) {
  const s = await redis.hGetAll(`session:${sid}`);
  if (!s.userId) throw new Error("session inconnue");

  // 1. Durée absolue depuis création (ex. 8h)
  if (Date.now() > Number(s.absoluteExpireAt)) {
    await redis.del(`session:${sid}`);
    throw new Error("session expirée (durée absolue dépassée)");
  }

  // 2. Inactivité : on remet le TTL à30 min àchaque requête réussie
  await redis.expire(`session:${sid}`, 30 * 60);
  return s;
}
```

Concrètement : si la session a été créée à 9h00 avec une durée absolue de 8h, elle expirera à 17h00 même si l'utilisateur reste actif jusqu'à 17h29. Cela évite qu'un attaquant ayant volé un cookie le 1er janvier puisse l'utiliser le 1er février.

3.2 Flags de cookies à connaître

Définition

Cookie : petit bloc de données stocké par le navigateur et renvoyé automatiquement à chaque requête vers le domaine d'origine. Mécanisme historique pour gérer les sessions web.

- `Secure` : cookie envoyé uniquement en HTTPS.
- `HttpOnly` : cookie inaccessible depuis JavaScript. Bloque le vol par XSS.
- `SameSite=Lax` (défaut récent) : couvre 95 % des CSRF.
- `SameSite=Strict` : maximum de protection mais navigation depuis un site externe ne porte pas le cookie.
- `SameSite=None; Secure` : pour cross-site légitime (auth fédérée).
- `Max-Age` ou `Expires` : durée de vie.
- `Domain` et `Path` : scope.
- Préfixe `__Host-` : impose Secure + Path=/ + pas de Domain (détaillé ci-dessous).

3.2.1 Le préfixe `__Host-` — la défense en profondeur des cookies

Définition

Préfixe `__Host-` : convention de nommage standardisée (RFC 6265bis) que les navigateurs interprètent comme un contrat de sécurité. Un cookie dont le nom commence par `__Host-` n'est accepté par le navigateur que si *toutes* ces conditions sont réunies : flag `Secure` présent, attribut `Path=/` présent, et `Domain` *absent*. Sinon le `Set-Cookie` est purement ignoré.

Pourquoi ces trois contraintes ?

- `Secure` : interdit l'envoi en HTTP clair, donc bloque l'interception réseau.
- `Path=/` : empêche l'*escalade par path* (un cookie scopé sur `/admin` qu'un attaquant ferait écraser depuis `/blog`).
- Pas de `Domain` : interdit le partage avec un sous-domaine. Un cookie sur `example.com` ne fuitera pas vers `partenaire.example.com`, ni vers une éventuelle XSS sur `blog.example.com`.

Exemple

Cookie `__Host-` accepté :

```
Set-Cookie: __Host-sid=8a2f...; Secure; HttpOnly;
          SameSite=Lax; Path=/; Max-Age=1800
```

Cookie `__Host-` rejeté (`Domain` présent) :

```
Set-Cookie: __Host-sid=8a2f...; Secure; HttpOnly;
          Path=/; Domain=example.com <-- rejeté
```

Cookie `__Host-` rejeté (`Path` différent de `/`) :

```
Set-Cookie: __Host-sid=8a2f...; Secure; Path=/api <-- rejeté
```

Cookie `__Host-` rejeté (pas de `Secure`) :

```
Set-Cookie: __Host-sid=8a2f...; HttpOnly; Path=/ <-- rejeté
```

Variante `__Secure-` : moins stricte, n'exige que le flag `Secure` (autorise `Domain` et `Path`). Utile quand on doit partager un cookie entre sous-domaines mais qu'on veut interdire HTTP.

4 Chiffrement au repos

4.1 Trois niveaux distincts

1. **Disque** : LUKS (Linux), FileVault (macOS), TDE pour BDD. Protège contre le vol physique (détaillé ci-dessous).
2. **Application** : chiffrement champ par champ (AES-256-GCM). Indispensable pour PII, données médicales, financières.
3. **Secrets** : Vault, AWS Secrets Manager, GCP Secret Manager. Jamais dans le code.

4.1.1 LUKS, FileVault, TDE — les chiffrements de stockage

Définition

LUKS (*Linux Unified Key Setup*) : standard de chiffrement de partition pour Linux, intégré au noyau via le module `dm-crypt`. Chiffre l'intégralité d'un disque ou d'un volume LVM avec AES-XTS. Au démarrage, l'utilisateur ou le système doit fournir une *passphrase* (ou utiliser une clé TPM) pour déchiffrer la partition. Une fois la partition montée, l'OS et les applications travaillent en clair — LUKS ne protège donc *que* contre l'attaque physique (vol du serveur).

Exemple : `cryptsetup luksFormat /dev/sda2` crée un volume LUKS. Puis `cryptsetup open /dev/sda2 secret_vol` pour le déverrouiller.

Définition

FileVault : équivalent macOS, intégré au système depuis 2003 (réécrit en FileVault 2 en 2011 avec AES-XTS). Le chiffrement est lié au compte utilisateur : seul son mot de passe peut déverrouiller le disque au démarrage. Une clé de récupération est stockée chez l'utilisateur ou dans iCloud.

Activation : *Préférences Système* → *Confidentialité* → *FileVault* → *Activer*. Aucune commande à apprendre.

Définition

TDE (*Transparent Data Encryption*) : fonctionnalité native des SGBD entreprise (Oracle Database, SQL Server, PostgreSQL avec extension, MySQL Enterprise). Chiffre les fichiers de données et les sauvegardes au niveau du moteur de base, sans que l'application n'ait à le savoir. Une clé maître stockée hors du serveur (KMS, HSM) déverrouille les clés de chiffrement à chaque démarrage.

Exemple SQL Server :

```
USE master;
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'P@ssw0rd!';
CREATE CERTIFICATE TDECert WITH SUBJECT = 'TDE Certificate';
USE Production;
CREATE DATABASE ENCRYPTION KEY
  WITH ALGORITHM = AES_256
  ENCRYPTION BY SERVER CERTIFICATE TDECert;
ALTER DATABASE Production SET ENCRYPTION ON;
```

Important commun aux trois. Ces mécanismes protègent uniquement contre le *vol physique* (disque sorti du serveur, sauvegarde dérobée). Tant que le système tourne, les données sont en clair en mémoire et dans les caches : un attaquant qui obtient un accès root sur le serveur lira tout. D'où la nécessité du chiffrement *applicatif* (niveau 2) pour les champs sensibles.

4.2 Envelope encryption

Définition

Envelope encryption : technique où chaque donnée est chiffrée par une clé éphémère propre (**DEK** = *Data Encryption Key*), elle-même chiffrée par une clé maître (**CMK** = *Customer Master Key*) qui reste dans un KMS (Key Management Service). On stocke à côté du chiffré la DEK déjà chiffrée par la CMK : pour déchiffrer, l'application demande au KMS de déchiffrer la DEK, puis utilise la DEK localement pour déchiffrer la donnée.

Pourquoi pas chiffrer directement avec la CMK ?

- Le KMS facture par appel (ex. AWS KMS : 0,03 $ pour 10 000 appels). Chiffrer chaque champ directement multiplierait la facture par le volume de données.
- Le KMS impose une taille maximale (4 KB pour AWS KMS) : impossible de chiffrer un gros document directement.
- Latence : un round-trip vers le KMS par champ ruinerait les performances.

Avec envelope encryption : un appel KMS *par session ou par lot* suffit pour obtenir une DEK qui chiffre localement des milliers de champs. La CMK ne quitte jamais le KMS.

Exemple

Chiffrement d'un champ `numero_carte_bancaire` avec AWS KMS :

```
import { KMSClient, GenerateDataKeyCommand, DecryptCommand } from "@aws-sdk/client-kms";
import { createCipheriv, createDecipheriv, randomBytes } from "crypto";

const kms = new KMSClient({ region: "eu-west-3" });

// ----- CHIFFREMENT -----
async function encryptCard(plain) {
  // 1. Demander une DEK au KMS (256 bits AES + sa version chiffrée)
  const { Plaintext: dek, CiphertextBlob: encDek } = await kms.send(
    new GenerateDataKeyCommand({
      KeyId: "alias/cards-master", // CMK référencée par alias
      KeySpec: "AES_256"
    })
  );

  // 2. Chiffrer la donnée localement avec la DEK
  const iv = randomBytes(12);
  const cipher = createCipheriv("aes-256-gcm", dek, iv);
  const ct = Buffer.concat([cipher.update(plain, "utf8"), cipher.final()]);
  const tag = cipher.getAuthTag();

  // 3. Stocker dans la base : IV + tag + ciphertext + DEK chiffrée
  return { iv, tag, ct, encDek };
}

// ----- DECHIFFREMENT -----
```

```
async function decryptCard({ iv, tag, ct, encDek }) {
  // 1. Demander au KMS de déchiffrer la DEK
  const { Plaintext: dek } = await kms.send(
    new DecryptCommand({ CiphertextBlob: encDek })
  );
  // 2. Déchiffrer localement
  const decipher = createDecipheriv("aes-256-gcm", dek, iv);
  decipher.setAuthTag(tag);
  return Buffer.concat([decipher.update(ct), decipher.final()]).toString("utf8");
}
```

Résultat en base :

```
+----+------------+------------------+------------------+
| id | iv (hex) | tag (hex) | ct (hex) | encDek (hex)
| 42 | a3f1... | 7c2e... | b1d4e8... | ABCD...0900...
+----+------------+------------------+------------------+
```

La CMK ne quitte jamais AWS. Un attaquant qui dump la base ne peut rien déchiffrer sans appeler le KMS, qui journalise et peut être révoqué.

4.3 Algorithmes recommandés

- Chiffrement symétrique : **AES-256-GCM** ou **ChaCha20-Poly1305**.
- Hash de mot de passe : **Argon2id**, **bcrypt**, **scrypt**, **PBKDF2**. Jamais SHA seul (détaillé ci-dessous).
- Asymétrique : RSA-OAEP (2048+), ECC P-256/P-384, Curve25519.

4.3.1 Les quatre algorithmes de hash de mot de passe — exemples

Pourquoi pas un simple SHA ? SHA-256 est conçu pour être *rapide*. Un GPU calcule des milliards de SHA-256 par seconde, ce qui rend trivial un brute force contre un mot de passe humain. Les algorithmes de hash de mot de passe sont au contraire conçus pour être *lents et coûteux en mémoire* : on les paramètre pour qu'une vérification prenne 50–500 ms (acceptable côté serveur, prohibitif côté attaquant).

Exemple

Argon2id (recommandé par OWASP en 2026, vainqueur 2015 du Password Hashing Competition) :

```
import argon2 from "argon2";
const hash = await argon2.hash("Sup3rSecret!", {
  type: argon2.argon2id,
  memoryCost: 65536, // 64 MB de RAM par vérification
  timeCost: 3, // 3 itérations
  parallelism: 4
});
// hash = "$argon2id$v=19$m=65536,t=3,p=4$<salt>$<hash>"
const ok = await argon2.verify(hash, "Sup3rSecret!");
```

bcrypt (standard de fait depuis 1999, encore largement déployé) :

```
import bcrypt from "bcrypt";
const hash = await bcrypt.hash("Sup3rSecret!", 12); // cost=12  ~250ms
// hash = "$2b$12$<salt22><hash31>"
const ok = await bcrypt.compare("Sup3rSecret!", hash);
```

scrypt (Colin Percival, 2009, intégré au module `crypto` Node) :

```
import { scryptSync, randomBytes, timingSafeEqual } from "crypto";
const salt = randomBytes(16);
const hash = scryptSync("Sup3rSecret!", salt, 64, { N: 1 << 15, r: 8, p: 1 });
// stocker : salt.toString("hex") + ":" + hash.toString("hex")
```

PBKDF2 (RFC 2898, 2000 — le plus ancien, le moins sûr aujourd'hui mais le mieux supporté en environnements contraints) :

```
import { pbkdf2Sync, randomBytes } from "crypto";
const salt = randomBytes(16);
const hash = pbkdf2Sync("Sup3rSecret!", salt, 600000, 32, "sha256");
// 600 000 itérations = recommandation OWASP 2023
```

Choix en 2026 : privilégier **Argon2id** pour un nouveau projet. **bcrypt** reste un excellent défaut si Argon2 n'est pas disponible. **scrypt** et **PBKDF2** en compatibilité ou contrainte légale (FIPS).

5 Les outils du module

5.1 Certbot

Outil

Certbot (`https://certbot.eff.org`) est un client ACME open source, édité par l'**EFF** (*Electronic Frontier Foundation*, ONG américaine de défense des droits numériques fondée en 1990). Standard de fait pour automatiser l'obtention et le renouvellement de certificats Let's Encrypt.

Commandes de base :

```
sudo certbot certonly --nginx -d api.example.com
sudo certbot renew --dry-run # tester le renouvellement
```

Exemple complet : nginx en *reverse proxy* HTTPS devant un backend Node.

1. Hypothèses. Domaine `api.exemple.fr` pointant vers la VM. Backend Node qui écoute sur `127.0.0.1:3000` (en HTTP, sans TLS, jamais exposé à Internet).
2. Installation :

```
sudo apt install nginx certbot python3-certbot-nginx
```

3. Première version de `/etc/nginx/sites-available/api` (HTTP seul, pour valider le challenge) :

```
server {
    listen 80;
    server_name api.exemple.fr;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/api /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

4. Obtention automatique du certificat et modification du nginx (le plugin `-nginx` édite la config tout seul) :

```
sudo certbot --nginx -d api.exemple.fr \
  --agree-tos -m ops@exemple.fr --redirect
```

Certbot va : (a) émettre un challenge HTTP-01, (b) obtenir le certificat (`/etc/letsencrypt/live/api.exemple.fr/`), (c) modifier le fichier nginx pour ajouter le bloc HTTPS et la redirection automatique HTTP→HTTPS.

5. Fichier nginx final (après passage de certbot) :

```
server {
    listen 80;
    server_name api.exemple.fr;
    return 301 https://$host$request_uri; # redirection forcée
}

server {
    listen 443 ssl http2;
    server_name api.exemple.fr;

    ssl_certificate /etc/letsencrypt/live/api.exemple.fr/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.exemple.fr/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    add_header Strict-Transport-Security
        "max-age=63072000; includeSubDomains; preload" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "no-referrer" always;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Connection "";
    }
}
```

6. Renouvellement automatique. Le paquet certbot installe un *systemd timer* qui exécute `certbot renew` deux fois par jour. Vérification :

```
systemctl list-timers | grep certbot
sudo certbot renew --dry-run # simulation
```

Au final : votre API Node ignore tout de TLS. Toute la sécurité du transport, du renouvellement de certificat et des headers est gérée par nginx en façade — exactement comme le ferait un CDN.

5.2 nginx

Outil

nginx est un serveur HTTP / reverse proxy / load balancer largement utilisé. Idéal devant une API pour gérer la terminaison TLS, les headers de sécurité, la compression, le cache, le routing.

5.3 testssl.sh

Outil

testssl.sh (`https://testssl.sh`) : script bash open source qui audite la configuration TLS d'un serveur. Détecte les versions, les ciphers, les vulnérabilités (Heartbleed, POODLE, BEAST), la configuration HSTS.

Exemple :

```
testssl.sh -p api.example.com
testssl.sh --vulnerable api.example.com
```

5.4 Qualys SSL Labs

Outil

SSL Labs (`https://www.ssllabs.com/ssltest`) : service en ligne gratuit de Qualys qui note la configuration TLS d'un site (de F à A+). Référence dans l'industrie. Permet aussi de connaître les défauts spécifiques.

5.5 Mozilla SSL Configuration Generator

Outil

Mozilla SSL Config (`https://ssl-config.mozilla.org`) : générateur de configuration TLS pour nginx, Apache, HAProxy, etc. Trois profils (Modern, Intermediate, Old). Indispensable pour ne pas se tromper.

5.6 HashiCorp Vault, AWS Secrets Manager

Outil

Vault (HashiCorp) et **AWS Secrets Manager** sont des gestionnaires de secrets : ils stockent et distribuent de manière sécurisée tokens, mots de passe, clés API. Permettent rotation, audit, révocation. *Indispensables dès que l'on a plus d'un environnement.*

6 Synthèse

À retenir

- RBAC pour 90 % des cas. ABAC quand la complexité l'exige. ReBAC pour les apps collaboratives.
- Refus par défaut, vérification à chaque endpoint et au niveau objet.
- TLS 1.2 + 1.3 uniquement. ECDHE obligatoire pour la PFS.
- HSTS long (1–2 ans), `includeSubDomains`, `preload`.
- Cookies blindés : `Secure + HttpOnly + SameSite + Max-Age`.
- Secrets dans un KMS, jamais dans le code. Envelope encryption pour les données sensibles.

7 Lexique du module

ABAC Attribute-Based Access Control.

ACME Automated Certificate Management Environment (RFC 8555).

AES-GCM Algorithme de chiffrement symétrique authentifié, standard moderne.

Argon2 Algorithme de hash de mot de passe, vainqueur du Password Hashing Competition.

Bcrypt Algorithme de hash de mot de passe, encore très utilisé.

CSPRNG Cryptographically Secure Pseudo-Random Number Generator.

Cipher suite Combinaison d'algorithmes utilisée par TLS pour une session.

CSRF Cross-Site Request Forgery.

Curve25519 Courbe elliptique moderne, utilisée par Ed25519 et X25519.

DEK Data Encryption Key. Clé éphémère utilisée pour chiffrer une donnée.

ECDHE Elliptic Curve Diffie-Hellman Ephemeral. Échange de clé moderne assurant la PFS.

ECDSA Elliptic Curve Digital Signature Algorithm.

Envelope encryption Chiffrement avec une clé maître + clés éphémères.

HSTS HTTP Strict Transport Security.

HTTPS HTTP sur TLS.

KMS Key Management Service.

Let's Encrypt CA gratuite et automatisée, lancée en 2016.

mTLS Mutual TLS. Authentification mutuelle par certificats.

OPA Open Policy Agent. Moteur de politiques open source.

PFS Perfect Forward Secrecy.

PEP/PDP/PIP/PAP Architecture d'autorisation : Enforcement / Decision / Information / Administration Point.

RBAC Role-Based Access Control.

ReBAC Relationship-Based Access Control.

Refus par défaut Principe : toute action non autorisée explicitement est interdite.

SameSite Flag cookie qui contrôle l'envoi cross-site.

SSL Secure Sockets Layer (ancêtre de TLS).

TDE Transparent Data Encryption (chiffrement disque BDD).

TLS Transport Layer Security.

Vault Gestionnaire de secrets HashiCorp.

Wildcard certificate Certificat couvrant tous les sous-domaines.

Zero-trust Principe : ne faire confiance à personne par défaut, même en interne.

8 Pour aller plus loin

- Mozilla *Server Side TLS* guide.
- Ivan Ristic, *Bulletproof TLS and PKI*.
- RFC 8446 (TLS 1.3), RFC 6797 (HSTS), RFC 8555 (ACME), RFC 9325 (TLS Best Current Practice).
- Google Cloud BeyondCorp (modèle zero-trust).